

What Retry Budgets Can Guarantee: Calls, Protected Work, and Partial Repair

ANONYMOUS AUTHOR(S)

A retry limit bounds failed attempts, but fallback can put all remaining computation under protection. We derive exact call/protection frontiers for persistent whole-kernel jobs. Cached completion meets a universal cap $Q \leq n + r$ and the best protected-work frontier for every unknown write budget B . Without it, the same entrypt contract forces all work under protection already at $B = r$. For repair work alone and a positive retry allowance, a workflow admits one optimum for all budgets exactly when every job has a one-write maximum damage. Call charges change this boundary; one-job loss bounds and workflow certificates expose the roles of precedence and history. Java operations realize the whole-kernel bounds under stated assumptions. Roslyn modifications preserve checked event, snapshot and compiler projections; stronger rebase operations retain merge costs outside these models. Authored arrivals show count tradeoffs and mixed timing effects. These results identify interfaces' guarantees and information requirements, with experiments delimiting their operational interpretation.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, partial repair, online policies

1 Introduction

In optimistic update loops, a caller prepares outside protection and validates before publication. Switching to protected execution after finitely many failures bounds failed calls, but can protect all remaining computation. Which bounds on calls and protected work can be met together, and which completion operations are necessary?

Bounded speculation followed by locking is established practice. glibc's enabled x86 elision path defaults to three transaction starts; HotSpot exposes RTMRetryCount [22, 23]. HTM can abort without a write and discards speculative work, so it motivates finite attempt budgets but does not instantiate our retained-preparation model. We study validate-and-complete interfaces with per-update, shared-batch or combined retry allowances. These specification choices yield different call/protection guarantees. No latency SLA is assumed.

Consider two dependent transformations with works 8 and 16 and a cap of two validation or completion calls. Every call must complete a transformation. Fresh protected completion performs all 24 units under protection. Cached completion instead prepares outside and, within one completing callback, either reuses the preparation or recomputes after a mismatch. Its worst protected work is 0, 16, or 24 when at most 0, 1, or 2 writes occur (Figure 1). The policy does not receive this bound. The benefit has a cost: at one write its worst total work is 40, compared with 24 for fresh protected completion.

For works (1, 2, 3, 5), a six-call cap allows $r = 2$ failures. Cached completion guarantees at most eight protected work units at four writes. With only fresh fallback, every policy satisfying that universal cap can be forced to protect all eleven units at two writes. Cached completion is necessary for this pair within the specified entrypt contract. The units count kernel work, not milliseconds.

If an update repairs only dirty components, the value of knowing B depends on write footprints. Charging each call can change simultaneous optimality again. Roslyn's prepared content-rebase paths illustrate stronger operations whose merge cost must be retained.

Our contributions are exact call/protection frontiers; partial-repair information boundaries with workflow and call-cost extensions; and an operational assessment of Java realizations, Roslyn preservation and timing. Fallback, online contention control, synchronization choices and robust

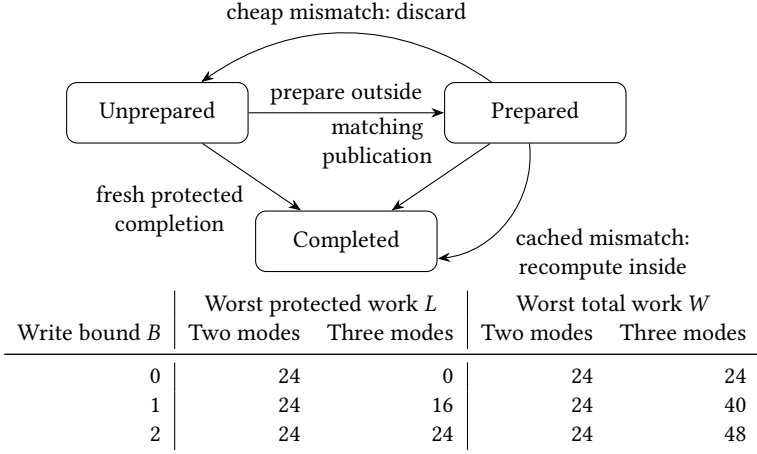


Fig. 1. Operation transitions and the two-job example with $r = 0$. A cached callback completes on either comparison outcome; a cheap mismatch requires a new preparation. Both fixed policies satisfy $Q \leq 2$ in every finite-write environment; neither receives B . Values follow from the model, not timing measurements.

dynamic programming are inherited [5, 8, 11, 12, 20, 21]. The additional claims concern these resource equations and their operation-specific lower bounds.

Three design decisions, three different guarantees. A library designer can compare operations against the six-call/eight-work contract. Its fixed protection ceiling needs a valid write bound; the policy and entire guarantee curve require no estimate. Here r limits retries and B counts external writes. Footprints determine the value of budget knowledge; call charges guide speculation. Source experiments check preservation and costs, without establishing adoption of these authored requirements.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. A state records live own-input identities, completed jobs and their immutable outputs, and a finite store of snapshots and prepared records. A record contains its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without the corresponding computation. In Table 1, U means no selected record, C a selected record, possibly stale, and D permanent completion. Failed cheap calls clear the selection, leaving retained copies available. After every finite history,

each unfinished job admits a legal own-key write with an identity never used for that job. Writes preserve completed outputs.

A finite visible history h lists program actions, returned values and completion events. The program chooses a ready job and operation, or an unprotected inspection, preparation or local record operation. After that choice, the environment chooses a finite word of legal writes, then the operation executes. This *gate* precedes comparison, including within a callback. The mode/job is fixed before the gate; any later record selection uses only the prior store. Comparison and completing computation/publication are indivisible to writes; protection ends on return. Inspection honestly returns the live entry. Preparation computes a whole kernel from a captured immutable snapshot and completed parents; old captured snapshots are also allowed. Fixing the capture point, its pure body has no live read or visible interaction. Writes after capture can be replayed in order at the next gate, preserving the returned record and every subsequent observation. Only that pure body is contracted; capture is not moved.

Let W count all foreground kernel work, L its protected part, and Q validation, publication and completion calls, including failures. Preparations, initialization and external-writer calls are excluded from Q . These are separate resources. Common call overhead is excluded from L . Proposition 4 charges it explicitly in this interface; Section 4.2 treats the distinct repair interface. Let $\Omega_k(T)$ sum the largest $\min(k, |T|)$ works in T . Write $\Omega(T)$ for the total; omitted sets mean J . Set $\Omega_0 = 0$.

Table 1. Whole-kernel operations. Every validation or callback adds one to Q ; preparation adds none. Entries show successor state and added (W, L) . Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Conditional publication	C	D, $(0, 0)$	U, $(0, 0)$
Validation-only callback	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

Call the first two validating operations *cheap*: they publish on a match and may return without completion on a mismatch. The label concerns kernel work, not elapsed time. Two modes provide cheap publication and fresh completion; three also provide cached completion. Failed validation leaves the live entry unchanged. Inspection and local record operations cost no Q, W, L and never publish or retain a guard. Direct acquire/release, protection surviving return, cross-job callbacks and outside completion are absent from this endpoint contract. These restrictions define the comparison; they do not assert completeness for arbitrary Java code or lock APIs. Each selected operation terminates. A play that never completes all jobs, including infinite free actions, has infinite objective; fairness of selected operations is an explicit assumption.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j , $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \leq n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B -independent strategy satisfying that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$. Every finite-write play against a fixed program is preserved by capping its environment at that play's write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

Inspection and retention lemma. After an operation on unfinished job i is chosen, one fresh own-key write invalidates every retained i -record: all identities precede the gate. Two environments can

reproduce the same concrete prefix and differ only by this next write, with the same supplied B when applicable. Thus honest free inspections do not remove that option when its write budget remains available. Our target adversaries below have an unconditional finite write bound on every play. If a program inspects or avoids a target forever against one of them, that very play already contradicts finite-write completion. These facts extend the particular lower bounds to the full retained-record/inspection class; they do not eliminate observations in general.

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes, let $k = \min(B - r, n) > 0$ and target those k heaviest jobs. Whenever the program compares a target preparation, place a fresh own-key write immediately before that comparison. Cheap calls then fail, and cached calls complete with that target's full work protected. Fresh completion already protects the target, without needing a write. Non-target jobs need no writes.

Here is why the adversary stays within B . Let f count target comparisons that did not complete, and p target jobs already protected. Each adversarial write either increases f or completes a cached target; hence the number of writes is at most $f + p$. Before another target write, $f \leq r$ and $p \leq k - 1$, so the next write still gives at most $r + k \leq B$. This includes the write causing an $(r + 1)$ st failure: the cap-violation witness itself is within B . Stop all writes then. Completion still needs n completing calls in addition to those failures, contradicting membership in $\mathcal{P}_3(B, r)$. Thus every target is protected, costing at least Ω_k .

Every chosen write uses a new identity, invalidating all retained target preparations. An inspection or completed callback holds no guard across the next comparison. All schedules used above stop writing after a finite bound; the completion requirement rules out avoiding a target forever. $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$. At $B = r$, only an informed two-mode program knows no write remains and may risk another cheap call. The unaware class must also be safe in extensions with another write, causing the boundary shift.

Readiness constrains when a target is attempted, but persistence requires every target eventually. Thus DAG edges do not change this frontier. They become consequential with repair operations and call costs (Section 4.3).

3.1 Local Caps and a Shared Cap Can Be Required Together

Suppose job i may incur at most r_i noncompleting calls, and the batch at most $s \leq R = \sum_i r_i$. These caps hold in every finite-write environment. Define the weighted subset value

$$A_c(B) = \max_{T \subseteq J: \sum_{i \in T} c_i \leq B} \sum_{i \in T} w_i,$$

allowing zero-cost items. This is a knapsack value; the contribution is identifying the write cost that activates each job's protection.

PROPOSITION 3 (LOCAL AND SHARED RESOURCE CONTRACTS). *The minimum worst protected work under both caps is*

$$U_2(B; \mathbf{r}, s) = \max\{A_r(B), \Omega \mathbf{1}[B \geq s]\},$$

$$U_3(B; \mathbf{r}, s) = \max\{A_{r+1}(B), \Omega_{(B-s)_+}\}.$$

One policy attains each curve simultaneously for every B : switch to guaranteed completion when either the current local allowance or the shared allowance is exhausted. Its worst call count is $n + \min(B, s)$.

PROOF. For a local lower bound, target a maximizing subset. Reject each target's cheap calls up to its local cap. In two modes another rejectable call is then universally inadmissible, so the target completes protectively using at most r_i actual writes. In three modes, one additional write invalidates its eventual cached comparison; an earlier fresh completion already pays w_i . Thus each target costs at most $r_i + 1$ writes. The shared lower bounds follow from Theorem 2, since adding local constraints cannot enlarge its policy class.

For the switching policy, if fewer than s failures occur, a protected job has exhausted its own cap: it requires r_i writes in two modes, and $r_i + 1$ in three. If s failures occur, two modes pay at most Ω ; in three, every protected completion still needs its own additional mismatch write, so at most $(B - s)_+$ jobs are protected. These two cases give the maximum above. A fresh preparation after every rejection makes the intervals disjoint. Distributing $\min(B, s)$ cheap failures among the local allowances attains the call bound; its maximum need not coincide with the protection maximum. $\square$

With only local caps take $s = R$, giving $U_2 = A_r(B)$ and $U_3 = A_{r+1}(B)$. Uniform $r_i = r \geq 1$ yields $\Omega_{\lfloor B/r \rfloor}$ and $\Omega_{\lfloor B/(r+1) \rfloor}$, with $Q \leq n + nr$. At $r = 0$ these become Ω and Ω_B , with $Q = n$. Reusing the rejected preparation at the threshold would lose the extra-write requirement of the three-mode

bound. A per-job preparation cap is therefore a different contract. Sharing the same total allowance R permits weakly less worst protected work than reserving local allowances; requiring local limits restricts that flexibility.

3.2 Choosing Speculation When Every Call Has a Cost

Let each counted call cost $\kappa \geq 0$ and minimize $C = L + \kappa Q$. Let $C^*(B, r)$ be the minimum worst cost over the full B -informed program class of Theorem 1, with only the shared cap. Mode choice remains *before* comparison; the partial-repair interface below allows a different observation/action order. Write $C(P, B)$ for policy P 's worst cost at budget B .

PROPOSITION 4 (LOSS OF THE GLOBAL-THRESHOLD POLICY). *The three-mode policy P_r that freshly prepares before each attempt and switches after r cheap failures to cached completion has*

$$C(P_r, B) = n\kappa + \kappa \min(B, r) + \Omega_{(B-r)_+},$$

$$0 \leq C(P_r, B) - C^*(B, r) \leq \kappa \min(B, r) \leq r\kappa.$$

The $r\kappa$ bound is sharp for this policy.

PROOF. The call and protection upper bounds follow from Theorem 2. They are attained together: reject the first $\min(B, r)$ cheap calls, then use remaining writes to invalidate cached completions of the heaviest jobs. Every competitor has a resource lower-bound execution with both $L \geq \Omega_{(B-r)_+}$ and $Q \geq n$. Thus $C^* \geq n\kappa + \Omega_{(B-r)_+}$ on that execution. For one job and $B = r + 1$, invalidating every comparison forces protected completion. Immediate cached completion is optimal at $\kappa + w$, while P_r pays $(r + 1)\kappa + w$. $\square$

The bound compares a specified policy with an informed comparator. A caller may instead want the best possible loss without ever learning B . For one job this question has an exact answer.

PROPOSITION 5 (EXACT ONE-JOB MINIMAX ADDITIVE LOSS). *For one job with work $w > 0$, let P range over all deterministic programs that receive no B and complete with $Q \leq r + 1$ in every finite-write environment. Then*

$$\inf_P \sup_{B \geq 0} (C(P, B) - C^*(B, r)) = \min\{(w - \kappa)_+, r\kappa\}.$$

Always-cached completion attains the first term for $r \geq 1$; P_r attains the second. Choose the smaller, without estimating B .

PROOF. The informed one-job value is $\kappa + \min(B\kappa, w)$ for $B \leq r$, and $\kappa + w$ for $B > r$. For the first case, cheap attempts and immediate cached completion give the two upper bounds. For a lower bound, count all noncompleting calls in a . While $a < B$, invalidate each selected usable comparison with at most one fresh write. Actual writes are at most a ; a cached completion can therefore be invalidated within $a + 1 \leq B$ writes. Fresh completion already pays w . If $a = B$ is reached, stop writing; any subsequent completion has at least $B + 1$ calls, regardless of its mode. For $B > r$, invalidating through the cap uses at most $r + 1 \leq B$ writes and forces protected completion.

Now consider an unaware program with $r \geq 1$ and $0 < \kappa < w$. Again invalidate each chosen usable comparison, and let $a \leq r$ count all noncompleting calls before its first fresh/cached completion. If fresh, witness budget $B = a$ gives regret at least $a\kappa + w - \min(a\kappa, w) \geq w$. If cached and $a < r$, one more write and $B = a + 1$ give regret at least $a\kappa + w - \min((a + 1)\kappa, w) \geq w - \kappa$. If cached and $a = r$, use $B = r + 1$ to get $r\kappa$. These are upper bounds on actual writes, not assumed equalities. Stop the adversary after at most $r + 1$ noncompleting calls; the cap and finite-write completion exclude an infinite avoidance run. Fresh identities invalidate retained preparations after mode selection, so free inspections do not change the argument.

Always-cached has worst regret $(w - \kappa)_+$ for $r \geq 1$. Substituting the informed curve above into Proposition 4 gives worst regret $r\kappa$ for P_r . Nonnegativity covers $r = 0$, $\kappa = 0$ and $\kappa \geq w$. $\square$

This is a capped rent-or-buy specialization of the classical ski-rental tradeoff [26]. For a serial threshold $t \leq r$, subtracting the inevitable completion toll gives cost $B\kappa$ for $B \leq t$ and $t\kappa + w$ otherwise. Beyond $B = r$, the call cap also forces the informed comparator to complete protectively. Threshold selection and regret are inherited ideas; the proof additionally covers inspections, retained records and fresh completion. When $0 < \kappa < w$, immediate cached completion minimizes loss iff $w \leq (r + 1)\kappa$; otherwise use all retries. With $w = 8$, $r = 2$, tolls 1 and 3 select retries and immediate completion, respectively. The units are supplied costs; arbitrary-workflow minimax loss remains open.

4 When Partial Repair Makes the Budget Valuable

Whole-kernel recomputation makes the first invalidation of a preparation as damaging as further invalidations of that preparation. Some software can instead reuse unaffected components. We now characterize when that operation change destroys a common optimum. We first define a single-job repair interface, then extend it to workflows. This is not a reduction of every Roslyn merge to component work.

Repair interface and observations. One job consists of m components with positive weights w_c . A write changes fresh versions on one nonempty footprint from a fixed finite family Γ . Footprints may overlap and repeat; each occurrence costs one write. Outside protection, preparation reads and computes every component anew. A protected call observes the dirty set D , the union of footprints since that preparation. It either accepts, repairs exactly D and publishes the job, or rejects and prepares again outside. An empty observation can complete with no repair. No partial publication, free dirty-set query, retained protection or cheaper completion operation is available. A fresh whole-job completion costs at least repairing any dirty set. Calls and preparations terminate.

The turn order is preparation, an environment word of footprints, observation D , then the accept/reject choice within that same counted call. No write intervenes between observation and decision. A visible history lists job choices, observations and decisions, including current D ; repeated writes and word lengths are hidden. Informed strategies additionally receive B . Completion is required within $q \geq 1$ calls in every finite-write execution. One preparation per call permits at most q preparations. The objective is protected repair work, excluding outside preparation and common call overhead. Let

$$G(b) = \max_{0 \leq t \leq b, \gamma_j \in \Gamma} \sum_{c \in \gamma_1 \cup \dots \cup \gamma_t} w_c, \quad W^* = w\left(\bigcup \Gamma\right).$$

Then $G(0) = 0$, G is nondecreasing and subadditive, and $G(m) = W^* > 0$. Unreachable components do not contribute to W^* . Singleton footprints give the prefix sums of weights sorted in decreasing order. A general footprint family need not have a cheap profile: computing these unions is part of the input-processing cost.

THEOREM 6 (EXACT VALUE AND INFORMATION BOUNDARY FOR REPAIR). *For a controller supplied with B , minimum worst repair work is*

$$K_q(B) = G(\lfloor B/q \rfloor).$$

Among controllers receiving no B that attain $K_q(B) = 0$ for every $B < q$, the least possible worst-work envelope is

$$T_q(B) = G((B - q + 1)_+).$$

One policy attains this envelope for all B . For $q \geq 2$, a single controller is optimal for every B if and only if $G(1) = W^*$. For $q = 1$, such a controller always exists.

PROOF. Put $k = \lfloor B/q \rfloor$. The aware policy accepts work at most $G(k)$ and otherwise rejects while a call remains. An observation with greater work requires at least $k + 1$ writes. Rejecting $q - 1$ such observations exhausts all opportunities for a q th: $q(k + 1) > B$. Thus forced acceptance on the last call still meets $G(k)$. This policy also satisfies the call cap outside its supplied budget, where the work guarantee is not asserted. Conversely, a word of at most k footprints realizes $G(k)$. Repeat it after every preparation. Any accepting call pays $G(k)$, and q repetitions use at most $qk \leq B$ writes. Fresh whole-job completion is no cheaper.

For the unaware lower bound, write once in each of the first $q - 1$ preparation intervals. After observation $j < q$, the history is compatible with only j writes in total. Accepting positive damage, or fresh completion, would violate that budget's zero optimum. Hence the policy must reject all $q - 1$ times. In its final interval spend up to $B - q + 1$ writes realizing $G(B - q + 1)$. No retry remains. The matching policy rejects its first $q - 1$ nonempty observations, then accepts; earlier empty observations complete for zero work.

Every simultaneous optimum meets the preceding zero-work requirements. If $G(1) < W^*$, let $k \geq 2$ be the first index with $G(k) > G(1)$, and set $B = q - 1 + k$. Then $1 \leq \lfloor B/q \rfloor < k$, so $K_q(B) = G(1) < G(k) = T_q(B)$. Conversely, if $G(1) = W^*$, both expressions are zero for $B < q$ and W^* otherwise. For $q = 1$ their arguments coincide. $\square$

The boundary concerns write damage: a footprint covering all reachable work preserves simultaneous optimality. With separate-component writes, further damage can make budget knowledge change the optimal acceptance threshold.

For two unit components, singleton writes and $q = 2$, $K_2(B)$ at $B = 0, 1, 2, 3$ is $0, 0, 1, 1$. Optimality at $B = 1$ forces an unaware policy to reject its first dirty component. Two subsequent writes make both components dirty; its last call costs 2 at $B = 3$. The aware policy at that budget accepts one dirty component immediately and retries only two-component damage. Figure 2 isolates the shared observation behind this separation.

Budget	First observation	Unaware decision	Possible final observation/cost
$B = 1$	Only component A dirty	Must reject to achieve optimum 0	Empty; cost 0
$B = 3$	Only component A dirty	Same observed prefix: reject	Both A and B dirty; cost 2

Fig. 2. Two unit components and two calls. Before the first decision the controller cannot distinguish these budgets. At $B = 3$, an informed controller can accept the first unit of damage and reject only full damage, guaranteeing cost 1. The unaware controller's forced first rejection exposes it to cost 2. No timing assumption is used.

Quantifying the loss without a budget. For repair work alone, a finite competitive ratio requires zero work when $K_q(B) = 0$. Theorem 6 therefore identifies the exact optimal ratio as $\sup_{B \geq q} T_q(B)/K_q(B)$. Subadditivity gives an upper bound q : with $k = \lfloor B/q \rfloor$, $B - q + 1 \leq qk$, so $G(B - q + 1) \leq qG(k)$. It is sharp for sufficiently many unit singleton components at $B = 2q - 1$, where the ratio is q . Thus the loss can grow with the call allowance; it is not a constant-price perturbation of whole-kernel optimality.

4.1 A Shared Retry Allowance Across a Workflow

The same boundary extends beyond a single job. Consider n persistent jobs with a DAG, each with its own component/footprint family. A ready job is freshly prepared before each attempted observation; no other preparation is retained. A write affects the currently attempted job, with no cross-job footprint. After a rejection the next ready job may be selected anew. The total call cap is $n + r$, with a shared r ; acceptance pays that job's dirty-component work and permanently completes it. Write $W_i = G_i(\infty)$, $\sigma_i = \min\{b : G_i(b) = W_i\}$, $\Sigma = \sum_i \sigma_i$, and $W^* = \sum_i W_i$. Let $K(B)$ be the known-budget minimum worst repair work for this interface.

THEOREM 7 (WORKFLOW INFORMATION BOUNDARY). *For $r \geq 1$, one policy simultaneously attains $K(B)$ for every B iff $G_i(1) = W_i$ for every job. This holds both for a fixed order and for choices among DAG-ready jobs. Under the condition,*

$$K(B) = \text{sum of the largest } \min((B - r)_+, n) \text{ values } W_i.$$

For $r = 0$, accepting every observation is simultaneously optimal for arbitrary footprints. More generally, the first budget at which $K(B) = W^$ is*

$$B_{\text{sat}} = \Sigma + r \max_i \sigma_i.$$

PROOF. If every job has a one-write maximum, reject the first r nonempty observations globally and otherwise accept. Early acceptances are clean; after r rejections, at most $(B - r)_+$ jobs can accept positive damage. To force the matching lower bound, target that many heaviest jobs, writing once before each target observation to cause maximum damage. A target completion costs W_i , and at most r rejections add writes, so $r + k \leq B$ writes suffice for k targets. An attempted $(r + 1)$ st rejection would itself violate the call cap within budget; truncate the adversary there. Selection and precedence do not avoid completing the targets.

For necessity, all $K(B)$ with $B \leq r$ are zero. A simultaneous optimum must therefore reject when one write is issued before each of its first r observations: after observation t , the history is compatible with only $t \leq r$ writes in total. No job has yet completed. Now spend σ_i writes to saturate each job on its next attempt. No rejection remains, so the policy pays W^* with $B_0 = \Sigma + r$ writes. If some $\sigma_j \geq 2$, a competing policy reserves all r rejections for fully dirty observations of job j and accepts everything else. Full total work would require $\Sigma + r\sigma_j > B_0$ writes, as every invocation uses a fresh preparation. Its worst cost is therefore strictly below W^* ; the attainable cost set is finite. This contradiction proves necessity even in a prescribed order.

At $r = 0$, every call must complete and acceptance dominates fresh whole-job completion. The value is $\max_{b_i \leq B} \sum_i G_i(b_i)$ independently of order. For the final statement, saturating every attempted observation uses at most $\Sigma + r \max_i \sigma_i$ writes on an admissible run and forces full work. Below that budget, reserve all r rejections for full observations of a maximum- σ_i job. Full work would require exactly that many writes, so the worst cost is strictly smaller. $\square$

For two jobs, one with two unit singleton components and one with a single unit component, $r = 1$ gives $K(B) = 0, 0, 1, 2, 2, 3$ at $B = 0, \dots, 5$ in either order. Yet optimality at $B = 1$ forces a first rejection; three subsequent writes saturate both jobs and force cost 3 at $B = 4$, where $K(4) = 2$. The shared allowance does not remove the information gap. The theorem identifies its existence and saturation threshold; it does not give a closed form for every intermediate value in an arbitrary nonsaturating workflow.

4.2 Charging Every Call Changes the Boundary Again

Repair work omits the protected cost of validation itself. To expose this assumption, charge $\kappa \geq 0$ on *each* call and a common $\mu \geq 0$ once upon completion. Acceptance on call t costs $\mu + t\kappa + w(D)$.

Preparation remains outside this metric. Let M be the sorted multiset of qm numbers

$$\{t\kappa + G(k) : 1 \leq t \leq q, 0 \leq k < m\},$$

with one-based entries M_j and repeated entries retained.

THEOREM 8 (TOLL CURVE AND SIMULTANEOUS-OPTIMALITY BOUNDARY). *The known-budget optimum is $\mu + C_q(B)$, where*

$$C_q(B) = \begin{cases} \min\{\kappa + W^*, M_{B+1}\}, & B < qm, \\ \kappa + W^*, & B \geq qm. \end{cases}$$

For $q \geq 2$, simultaneous optimality holds exactly in the following cases: $\kappa = 0$ and $G(1) = W^$, or $\kappa \geq W^*$. It fails for every $0 < \kappa < W^*$. The common cost μ does not change these conditions. At $q = 1$, first acceptance is always simultaneously optimal.*

PROOF. Remove μ and fix $T < \kappa + W^*$. On call t , an unacceptable dirty set has $t\kappa + w(D) > T$. Its least write cost is $a_t(T) = \min\{k : t\kappa + G(k) > T\}$. Such an index exists since $G(m) = W^*$. A sequence of q unacceptable observations is possible exactly when $\sum_t a_t(T) \leq B$: each interval can independently realize a minimum-cost witness. Otherwise accepting the first acceptable observation finishes within q calls and cost T . Because G is nondecreasing, $a_t(T)$ counts that shifted list's entries at most T . Hence $\sum_t a_t(T)$ is the rank of T in M , with ties. The smallest achievable threshold below the first-accept cap is M_{B+1} , proving the curve.

If $\kappa > 0$ and a policy ever rejects a first observation, follow it by maximum damage until it accepts. At a sufficiently large finite budget it pays at least $2\kappa + W^*$, exceeding the aware first-accept cap $\kappa + W^*$. Thus a simultaneous optimum must accept every first observation. Let k^* be the first index with $G(k^*) = W^*$. With known budget k^* , accept all first observations except maximum damage; reject that one and the next observation is necessarily clean. Its worst cost is at most $\max\{\kappa + W_<, 2\kappa\}$, where $W_< < W^*$ is the largest attainable nonmaximum damage. For $0 < \kappa < W^*$ this is strictly smaller than the first-accept value $\kappa + W^*$. Conversely, when $\kappa \geq W^*$, presenting a first observation with damage $G(B)$ forces either $\kappa + G(B)$ on acceptance or at least 2κ on rejection. First acceptance attains the former, which is no larger. The zero-toll case is Theorem 6. $\square$

Even a small positive call charge precludes a common optimum until it reaches maximum repair work. This sensitivity identifies the objective for which ignoring overhead preserves exact optimality.

Whole-kernel operations permit retained records and free inspection. Repair reveals damage only within each call and freshly prepares at most $n+r$ times in a workflow. Equal curves under one-write saturation follow from separate proofs, without equivalence of the games. Competitive-threshold compilation and its full outcomes remain in the supplement.

4.3 With Call Charges, Precedence and History Matter

Return to the workflow interface of Section 4.1, charging $\kappa > 0$ per call. Write $K^\kappa(B)$ for its known-budget value, $H(B) = \max_{\sum b_i \leq B} \sum_i G_i(b_i)$ and $C_{\max} = n\kappa + \sum_i W_i$.

THEOREM 9 (CALL CHARGES IN WORKFLOWS). *For independent jobs and $r \geq 1$, a simultaneous optimum exists iff $\kappa \geq \max_i W_i$. This condition is sufficient on every DAG: first acceptance has exact value $n\kappa + H(B)$. On a general DAG, a sink with $W_i > \kappa$ prevents simultaneous optimality. The condition on all jobs is no longer necessary.*

PROOF. For sufficiency, fix an allocation attaining $H(B)$. Present its dirty set on each job's first attempt and leave that job clean after a rejection. Accepting pays the allocated damage; rejecting incurs an extra toll at least as large. This matches the first-accept upper bound, regardless of order.

For necessity, a simultaneous optimum cannot reject after a prefix whose completed jobs were all fully dirty: saturating every remaining attempt would then force cost above $C_{\max}$ at some finite budget. Hence fully dirty first observations force it to pay $C_{\max}$ with Σ writes. If a job j with $W_j > \kappa$ can be placed last, an aware policy at budget Σ accepts preceding jobs. If any observation is less than full it accepts throughout, costing strictly below $C_{\max}$. Otherwise all Σ necessary writes are spent; rejecting j once then completing cleanly costs $C_{\max} - W_j + \kappa < C_{\max}$. The finite outcome set makes the worst cost strictly smaller. Every independent job, and every DAG sink, can be last.

For the final claim, let A have singleton components of weights 1 and 2, let C have one component of weight 1, require A before C , and take $r = 1, \kappa = 2$. First acceptance has curve 4, 6, 7, 8 at $B = 0, 1, 2, \geq 3$. Matching lower bounds subtract the mandatory toll 4. At $B = 1$, dirty A 's weight-2 component: acceptance or rejection costs at least 2. At $B = 2$, do the same, then dirty C after acceptance (cost 3), or the weight-2 component again after rejection (cost 4). At $B = 3$, first saturate A with two writes; then dirty C after acceptance (cost 4), or the weight-2 component after rejection (cost 4). Thus first acceptance is a common optimum although $\kappa < W_A$. Reversing the order makes A a sink and removes every common optimum. $\square$

History-dependent rejection can be essential even with one-component jobs. Consider chain (a, h, a) , $h > a > 0$, with $r = 1, \kappa = a$. Accept throughout except that a dirty middle job is rejected once if the first job completed cleanly. Its exact curve is $3a, 4a, 4a + h, 5a + h$ at $B = 0, 1, 2, \geq 3$. First acceptance instead costs $3a + h > 4a$ at one write. For upper bounds, a retry leaves at most $B - 1$ writes; whenever its extra toll a is paid, the clean first job has saved a . For matching lower bounds, one dirty observation forces at least a beyond baseline. With two writes, leave the first job clean and dirty the middle, then dirty the last after acceptance or the middle again after rejection, forcing $a + h$. With three writes, dirty the first job: after acceptance apply that two-write strategy to the last two jobs; after rejection dirty the middle and last instead. This forces $h + 2a$. A preceding clean rejection already spends the toll and leaves no retry. No writer budget is supplied; $(a, h) = (1, 3)$ gives the saved seven-case separation.

Constructing a Common Policy for Other DAGs. A finite recurrence decides whether a common optimum exists beyond these cases. Let $c(D)$ be the minimum footprint-word length realizing observation D on the attempted job. Set $E = \Sigma + r \max_i \sigma_i$; saturating every attempt proves $K^\kappa(b) = C_{\max}$ for $b \geq E$. Let e be the least-consistent write count clipped to E , t the remaining rejections, and a the cost already paid. Define $\Theta(\emptyset, t, e) = K^\kappa(e)$ and, with $e_D = \min(e + c(D), E)$,

$$\Theta(S, t, e) = \max_{i \in \mathcal{A}(S)} \min_D \max \{ \Theta(S - i, t, e_D) - \kappa - w_i(D), \Theta(S, t - 1, e_D) - \kappa \}, \quad (1)$$

omitting the rejection term at $t = 0$. A continuation exists exactly when $a \leq \Theta(S, t, e)$; in particular, $\Theta(J, r, 0) \geq 0$ decides common optimality.

To prove this, consider a completed history and sum the minimum footprint-word lengths of its observations. Fresh preparations and independent own-job writes realize those words in separate intervals, so the sum is the least budget consistent with that history. Every larger budget permits the same history by leaving writes unused. Since K^κ is nondecreasing, meeting its value at the least budget is necessary and sufficient for that history to meet every consistent target. Above E all targets equal $C_{\max}$; clipping this coordinate is exact even though a partial observation's minimum word can exceed its job's full-saturation count. Accumulated cost a is never clipped.

Induct on $|S| + t$. With no jobs left, feasibility is the interval $a \leq K^\kappa(e)$. For a fixed ready job and observation, accepting adds $\kappa + w_i(D)$ and rejecting adds κ ; either strictly decreases the induction measure. Each child's feasible costs therefore give a shifted downward interval. The choice after observing D takes the larger endpoint. Requiring success for every observation takes

the minimum; choosing a job before that observation takes the maximum. This proves Equation (1) with its required $\exists i \forall D \exists \text{action}$ order. A maximizing job and any satisfied child inequality supply an executable policy using history alone. If the initial interval excludes zero, the failed alternatives identify an adversarial finite observation path for each attempted controller choice.

For each minimum footprint count, only its maximum attainable damage is needed in the adversarial alternatives. With I unfinished sets and $h = \max_i (m_i + 1)$, the table has $O(I(r+1)(E+1))$ entries and uses $O(In(r+1)(E+1)h)$ arithmetic operations, given profiles and the comparator. For a fixed order, $I = n + 1$ and there is one job choice. This removes accumulated-cost enumeration, not potentially exponential DAG states or footprint profiling. It is a specialization of finite minimax and information-state reasoning, not a new general dynamic-programming principle.

5 Actual External Writes in Java

Use Java 17's `ConcurrentHashMap` with one foreground thread, distinct nonremoved keys, non-null immutable identity-equal values and never-reused write identities. Each job updates its fixed key using own snapshots and captured parents. Kernels and key hash/equality/comparison methods are stable, side-effect-free and normally terminating. Replacement accepts identity or equals equality [15, 16]. The API specifies atomic compute with one callback invocation [14]; the inspected source executes existing-key callbacks under a bin monitor, including tree bins [16]. Different keys may share bins.

Implement Table 1 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks never directly or recursively modify the map, as required by the API. Our model additionally excludes other-key reads, other-job kernels and guard work; multiple preparations remain allowed. Q counts API invocations, not monitor acquisitions; L counts only kernel work under protection. Internal transfer/treeification preserves stored value references and adds no modeled external write or kernel work. The monitor may be released before API return: abstract postpublication internal steps away and place intervening writes at the next gate. Callbacks retain no guard across calls. Scheduling and normal call completion are assumptions, not consequences of finite writing.

THEOREM 10 (RESOURCE BOUNDS FOR ACTUAL WRITES). *Under the stated API, monitor and scheduling assumptions, Theorems 1–2's exact call/protected-kernel-work bounds hold for Java programs using the listed operations, with separate or shared internal bins. This is a mathematical refinement argument for those operations, not a mechanically verified property of an entire JDK binary.*

PROOF. For the upper bounds, the serial policies freshly read and prepare after every cheap failure. Their read-to-comparison intervals, including cached completions, are disjoint; every mismatch contains a distinct successful write. Thus at most B mismatches occur. Every fresh/cached completing call completes exactly one job, and a cached mismatch performs precisely that job's kernel inside.

For the lower bounds, use the target-job adversaries of Section 3. Immediately before a chosen comparison acquires its monitor, a successful own-key write installs a fresh identity, invalidating every retained preparation for that job. The writer need not run inside the monitor: moving this write before acquisition preserves the selected operation and its mismatch outcome. A cheap call fails; a cached call recomputes inside; a fresh callback already incurs its whole kernel inside. Stale copies cannot complete cheaply, captured parent outputs persist, and no monitor survives the preceding call. Bin collisions therefore do not prevent the next chosen write. The prohibition on

other-job work within a callback ensures it cannot complete several targets for one call. The same write counts and hypothetical extra-write argument apply. $\square$

Operational accounting. Native kernels fold captured predecessor seeds and an immutable array of length $8w_i - |\text{Pred}(i)| > 0$, executing exactly $8w_i$ selected operations. Counters separately record protected operations and calls. Allocation, waiting, initialization, writer work and other API instructions are excluded. Supplementary scalar charges are specified costs, not inferred timings.

6 Source Integrations and Their Boundaries

Roslyn document updates without a writer budget. The `Workspace.cs` loop at commit 6c4a46a3 transforms a captured immutable solution outside a semaphore, validates identity under it and retrieves mismatches. Publication preserves notification ordering [6, 7]. We modify only the `SourceText` path. After r global failures, two-mode completion transforms the latest solution inside; three-mode completion prepares outside, then reuses on match or recomputes inside on mismatch. The selector receives no writer/price tables.

Replayability and safety under protection are separate obligations. The scratch document list must be cleared each attempt; source review discusses replay and event ordering [7]. Our initial two-mode patch deadlocked on same-text notification reentry; restoring the pre-lock identity no-op repairs it. Four later missing-document cases timed out when virtual `GetDocumentName` reentered an unrelated update while holding the semaphore. Patch04 detects an absent target and runs the original transformation on the same captured solution after releasing protection, preserving errors/exception handling. All 14 fixed error cases then complete. These defects belong to our patches.

RQ2 checks linked propagation, event payloads and snapshots. Variable/lazy work, allocation and `VersionStamp` generation limit transfer to counts. Custom loaders, changed-text notification reentry and arbitrary compiler behavior remain outside scope. No upstream numerical call cap or latency SLA was found; r is authored. A stronger prepared-rebase comparator reuses existing `WithDocumentContentsFrom` when solution ID, related-document IDs and all affected project states remain unchanged. It merges prepared contents into the latest solution under protection, otherwise using three-mode fallback. This partial operation lies outside our whole-kernel interface.

7 Formal Corroboration and Operational Evaluation

RQ1 corroborates resource and information distinctions; RQ2 checks source preservation, counts and timing. Agreement with a theorem does not establish a needed application contract or speedup. All studies are authored exploratory tests with inputs fixed before execution; saved-arrival reanalysis is explicitly post-outcome. All failures, timeouts, invalids and corrections remain archived.

An Apple M2/24 GiB runs Python 3.10.9/3.12.14 and OpenJDK 17.0.19; inspected map source matches the cited version but its binary was not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, checkers, source comparison, proofs, analysis and writing; Claude supplied separate proof/direction critiques and manuscript reviews. Measurements are archived executions. These are author-side checks, not mechanical proofs, independent third-party reproduction or human studies.

An *input* fixes a graph, works and prices/footprints; a *root* adds budget and retry allowance. Native *maxima* range over fixed finite environments. The proofs establish unrestricted bounds; Table 2 does not sum overlapping studies.

Table 2. Bounded mathematical and native-count checks. Each row has its own denominator; earlier inputs may recur. Separate code reconstructs the compared quantities. These counts corroborate proofs and implementations, not external application validity.

Obligation	Evidence and comparison
Whole-kernel frontier	Direct equations on 265,629 budget/slack pairs from 5,421 DAGs. Native aware/unaware selectors: 9,314/17,714 records, 1,024/1,280 finite maxima.
Local/shared and fixed toll	Direct games: 35,796 local/shared-cap roots and 42,084 call-toll roots; canonical policies and separate bounds agree.
Single-job repair/toll	Repair: 3,495 inputs/160,080 checks. Per-call toll: 143,226 known-budget values and 33,688 competitive checks. Exact dirty-set games compare with curve/certificate methods.
Shared-retry workflows	8,960 graph/footprint/slack conditions, 61,246 known-budget roots. A quantified one-policy search on 6,208 conditions agrees with the all-budget existence criterion.
Call-charged workflows	37,248 conditions/216,282 budget roots, with a separate one-policy existence search on every condition.

7.1 RQ1: Bounded Corroboration of the Proofs

Native unaware selectors receive no budget, price or table state and test budgets through four; separate counters reconstruct W, L, Q . Earlier suffix/work refinements remain archived: gains were rare, with counterexamples to general work optimality and simultaneous intermediate-state optimality.

For the new shared-retry study, all ordered pairs of 24 one-/two-component footprint types cover an empty DAG and either chain direction, with $r = 0, 1, 2, 3$. Ordered triples of four representative types cover all eight forward-edge DAGs. The known-budget game chooses a ready job, observes a dirty set, then chooses acceptance or rejection. A separate existence search quantifies one policy over all dirty histories, using accumulated work and minimum consistent writes; it does not read the theorem’s saturation predicate as its answer. All 8,960 conditions and 6,208 existence tests agree, including the nonsaturated two-job separation. Four corrupted profiles are rejected. The positive-toll extension checks all 37,248 common-policy conditions, including four further corrupted profiles and seven cases where conditional rejection is required. All initial execution failures and corrections are retained in the artifact. This is bounded corroboration of the proof and its information pattern, not a new application population.

Direct action-before-comparison games also agree on all 35,796 local/shared-cap roots, including 8,928 known-budget local-cap comparisons, and 42,084 fixed-toll roots. Each study detects four changed-input/result controls. These games cover serial fresh preparations; the proofs supply the broader program-class lower bounds. A separate one-job comparison checks 11,232 budgets and 131,040 policy-budget values against direct informed games and complete serial policies; all agree with Proposition 5. Four changed-output/cap controls reject.

The threshold representation matches all 37,248 observed existence answers and 120 additional three-component cases. A checker importing no constructor verifies 123 exported tables from full dirty sets and rejects four changed certificates; exhaustive executable traces check two policies. It uses 803,712 states versus the boolean search’s 616,039: eliminating accumulated-cost enumeration does not ensure fewer states. Partial observations can require more writes than full saturation; only the comparator coordinate is clipped. The workflow profile controls individually alter $G(0)$,

maximum damage, saturation count or one observation's minimum write count by one. They target those consistency obligations, without estimating coverage of arbitrary checker defects.

Auxiliary repair compilation retains 21,028 small/rational comparisons, six initial scale timeouts (two compact-method), a changed search's 168 completed units and 4,220 additive-excess outcomes. These assess supplied footprints, without calibrating Roslyn merges or establishing inexpensive general profiling.

A controlled operational consequence. A Java probe uses four dependent kernels $(1, 2, 3, 5)s$, $r = 0, 1, 2$, and exactly $B = r$ initial failures. Both unaware policies have $Q = 4 + r$, $W = (11 + r)s$; two/three modes protect 11s/0 iterations. A budget-aware two-mode policy could also avoid protection. During every completing callback, a background compute on another key signals readiness before entry; the foreground holds its bin until that signal. Shared/disjoint-bin layouts have the same hook. Four scales, three allowances, two layouts/modes and three JVMs give 4,320 measurements/1,440 warmups; all complete and match 12 output references. A cross-job checker rejects 12 corruptions. At the largest scale, summed waits fall from 26.8 ms to 31.2 μ s in shared bins. Disjoint differences are submicrosecond and sometimes reverse. All 24 cells remain. This forced collateral-blocking probe is not natural-arrival performance or a latency bound.

7.2 RQ2: What Does the Source Integration Preserve?

Source projection and semantic checks. Pinned net8 Roslyn assemblies use a pinned .NET 10 RC runtime. Four projects' evaluated Compile items yield 3,837 files, 4,019 document occurrences and 40.8 MB, excluding two package-external items before outcomes. Fixed targets maximize sharing then size (63,593 bytes, two projects) and single-project size (background: 977,535 bytes). This source projection has internal references; updates append comments.

The semantic grid covers retry caps, concentrated/distributed writers, same/linked targets, no-ops, equal-content distinct texts and absent/deleted targets. Patch04's 174 cases plus 58 rebase cases all pass: 236 cases, 2,190 events, 1,253 publications and 2,653 snapshots. A separate checker binds immediate/queued payloads and callbacks to their solutions, checks linked publication chains and preserves 4,016 untouched text identities/versions and checked metadata. All 12 corruptions are rejected. Cases with $B = 5, 6$ use concentrated writes selected before outcomes; their schedules differ from earlier cases.

Reentry/same-text controls pass 18/5 cases for rebase and 14/4 for patch04. The original same-text deadlock and four missing-document timeouts remain recorded. A separate 96-process study changes types/constants, unrelated source, parse options, references or linked text across eight arms and cold/warm caches. All 2,556 project projections match declared source, symbol, diagnostic and emission expectations; eight corruptions reject. Workspace/direct compilations share Roslyn, and the declared expectations cover only these authored programs. These checks support the listed preservation obligations, not arbitrary concurrent equivalence.

Earlier timing and stronger source operations. Earlier forced and independent-arrival studies retain full denominators and outcomes. Three-mode foreground medians improved in four settings and worsened in seven. In 320 shared- $r = 2$ arrival batches, two/three modes used 10,802/10,786 calls and 7,131/3,641 protected transformations; three modes raised total transformations to 14,427. Background p95 improved at 100 μ s (60.00/306.10 μ s), but burst foreground time worsened (993.54/603.88 μ s).

Prepared rebase retains 4,219/4,528 protected merges at $r = 0/2$. A multi-document version passes the same semantic, reentry and compiler cases; its 1,728 arrival units give mixed batch/sequential ratios: 0.906–1.189 foreground, 0.511–1.404 background p95 and 0.917–1.281 protected spans. A finer

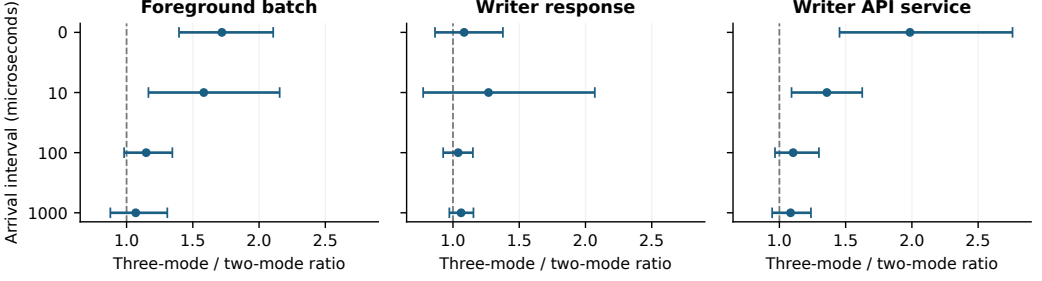


Fig. 3. New shared-cap timing comparison, $r = 2$. Points are geometric means of eight paired block ratios of process medians; bars are 95% percentile intervals from 10,000 fixed block-bootstrap resamples. Ratios above 1 favor two modes. Writer response is intended arrival to return; service is invocation to return. Each batch contributes its median across 64 writer requests. All four intervals and three metrics are shown.

guard changes four reference-removal decisions to reuse, with all 96 compiler outcomes agreeing and timing unmeasured. These stronger operations retain merge cost outside both idealized models.

Comparing shared and per-update contracts. The local baseline resets the retry field before each update in unchanged patch04 libraries. Eight arms comprise unmodified baseline, instrumented original, shared two/three at $r = 2$, and per-job two/three at $r = 1, 2$. Eight cyclic Latin blocks balance process positions. Each process serves 32 serial updates and 64 FIFO writer requests with precomputed texts, no supplied budget, and intervals $d \in \{0, 10, 100, 1000\} \mu s$. Each interval has two warm-up and ten measured batches, shuffled before execution. All 3,072 units (2,560 measured/512 warm-up) pass source checks and all 12 corruptions reject. This is an authored workload, not a deployment population.

For an update’s actual write count B_i , local modes satisfy $Q_i = 1 + f_i$, $f_i \leq \min(B_i, r)$, and inside count at most $1[B_i \geq r]$ (two) or $1[B_i \geq r + 1]$ (three). Per-update deltas sum to batch counts. All updates and batches meet Proposition 3’s count bounds. Table 3 reports every interval with separate medians, which need not coexist on one execution; B_{fg} counts foreground-period writes.

Table 3. New Roslyn cap comparison, 80 measured batches per cell. Entries are median $Q/B_{fg}/N_{inside}$: counted calls, foreground-period writes and full protected transformations. The universal call cap differs across rows.

Policy	Q cap	$d = 0$	10	100	1000 μs
Shared two, $r = 2$	34	34/3/32	34/3/32	34/2/31	33/1/0
Shared three, $r = 2$	34	34/34/30	34/27/22	34/8/6	33/1/0
Local two, $r = 1$	64	62/31.5/30	54/24/22	40/9/8	33/1/1
Local three, $r = 1$	64	58/64/25	53/50.5/19	37/7/1	33/1/0
Local two, $r = 2$	96	79.5/64/23.5	77/57/22	39/7.5/1	33/1/0
Local three, $r = 2$	96	66/64/17	64.5/64/16	38/7/0	33/1/0

At 100 μs , local three-mode $r = 2$ uses median 38 calls and zero protected transformations; shared $r = 2$ has 34 and 6. Their caps are 96 and 34: fewer protected updates permit more calls, rather than dominance under equal constraints. B_{fg} depends on completion time. Post-outcome reconstruction verifies that coordinate and the count bounds on 7,392 earlier batches; these comparisons have no common controlled write budget.

For shared three/two, foreground ratios at $0/10 \mu s$ are 1.72 [1.40,2.11] and 1.58 [1.17,2.16]; the two slower-arrival intervals include 1. All four writer-response intervals include 1 (Figure 3). Added per-update counter snapshots occur inside foreground batch duration. Original/baseline point ratios are 0.83–1.08 and all four intervals include 1, so this run does not identify instrumentation overhead; earlier harness estimates remain separate. Block resampling does not remove residual host/order effects, and no multiplicity correction is applied. Native execution took 237.84 seconds after the prior public replay completed, and the artifact includes setup/build and analysis costs. These results connect cap scope to checked counts while limiting any timing interpretation.

Earlier auxiliary objectives. Archived supplied-price, calibration and Deephaven studies retain all definitions and outcomes. Scalar gains occurred in 2/96 cells; timing fits met the fixed threshold in 2/8 settings. All-budget compilation sometimes cost more than requested-value computation; a phase-aware Deephaven model improved a previous abstraction. These adverse results concern auxiliary objectives, not the common-policy repair interface.

8 Related Work and Research Lineage

Fallback and online contention management. Kung and Robinson [5], Section 3.3/p.220, already retain the critical-section semaphore during reexecution after repeated validation failure. Lock-elision retry caps likewise precede our work [22, 23]. Cached completion here requires an unrolled-back preparation and atomic identity-check/recompute; HTM capacity or explicit aborts need not consume a write [24]. These mechanisms motivate the interfaces but do not establish their joint Q, L guarantee.

Attiya et al. [20], Sections 2–3/5, analyze one Greedy contention manager that does not receive the per-job non-conflict failure bound k . It is $O(ks)$ -competitive for makespan with s resources, with a corresponding deterministic lower bound under time-dependent resource requests. Thus a single policy for an unknown disturbance level and an all-policy lower bound are established. Our allowance counts global external writes, the cap constrains every finite-write execution, and the output is an exact protected-kernel-work curve for persistent jobs. Alistarh et al. [21], Sections 4–5, optimize whether and when to abort a conflicting transaction with known abort cost and unknown remaining duration. Their original throughput policy has no progress guarantee; Section 7 adds a probabilistic one. That objective and returned guarantee differ from the universal call cap here.

Guerraoui, Herlihy and Pochon [3], Sections 3–4, establish a priority-based abort/wait manager with bounded-delay progress and a makespan guarantee against an offline scheduler. Diegues and Romano [4], Sections 4–6 and its errata, adapt attempt budgets and capacity-abort handling using per-atomic-block measurements in GCC. Thus theoretically justified contention control and workload-oblivious adaptive fallback are prior contributions. Their transaction progress/cycle objectives differ from the persistent-preparation operations and hard local/shared count contracts here; their measurements do not calibrate our call toll.

Synchronization synthesis and robust control. Černý et al. [11] choose retries, guards and caches in partial concurrent programs with a scheduler and weighted limit-average safety objective. Their Theorem 6 uses quantitative probabilistic bisimulation; the erratum corrects complexity to PSPACE. Later work optimizes protected-statement counts and statistically profiles acquisition/contention [12]; preemption-safety synthesis also addresses deadlock-free lock placement [13]. Our archived scalar potential concerns initial worst total cost, not source equivalence or intermediate-state quotients. Finite n, B, r instances can be encoded as general games; we do not claim such tools cannot express them.

Mannor et al. [8] supply global-deviation-budget DP; Postek et al. [17] provide finite-candidate DP and causal budgeted routing. Information-state minimax control [25] is a general framework

for the repair stopping problem after proving its least-consistent-budget reduction. Our extra repair argument uses independently realizable intervals and monotonicity to reduce a feasibility tree to one minimum-write bad path. Generic backward induction and sufficient information are not new claims.

Online algorithms with predictions. Purohit, Svitkina and Kumar [27], Algorithm 2/Theorem 2.2, use predicted ski days and a trust parameter to trade consistency against robustness. Lykouris and Vassilvitskii [28], Sections 2–3, use next-request advice for caching, bounding expected misses against the offline sequence optimum across prediction errors. A single algorithm handling unknown prediction error is established. Here no advice is supplied: one deterministic policy must equal the B -informed minimax value for every B under a universal call cap. The additional claim is the one-write-saturation characterization for this repair game, not a new framework for online uncertainty.

Incremental computation and repair costs. Mokhov, Mitchell and Peyton Jones [1], Definitions 2.1/3.1 and Sections 4–6, classify build schedulers and rebuilders using persistent dependency/value information; Section 6.6 explicitly bounds iterative execution. Reuse, restarting and finite iteration are inherited concepts. Ley-Wild, Acar and Fluet [2], Theorems 4–5, relate consistent change propagation to trace distance and its cost. Repair-cost semantics is therefore also prior work. Our additional question is when to accept an observed dirty set or spend a limited call on fresh preparation, against an unknown external-write budget. Their stated store-consistency/minimality and trace-distance guarantees do not give this acceptance value or the common-policy existence boundary.

Inherited auxiliary results. Top-weight sums and allocation are familiar primitives [9, 10]; our bounds identify their operation-dependent write costs. The zero-fee snapshot [18] is an *unpublished, unreviewed author artifact* of auxiliary priced compilation. Its floor/cap, ordering, hardness and all adverse outcomes remain archived.

9 Limitations and Threats to Validity

The whole-kernel frontier excludes replenished budgets, structural change, exceptions, external completion and retained guards. Retained charged guards improve five of 384 inputs (including 242 versus 243); retaining protection beats immediate completion/release at one intermediate state (43 versus 45). General equality remains unproved.

Roslyn covers selected paths and authored changes. Our deadlocks/timeouts separate replayability from safe protected execution. Checked projections share Roslyn and do not establish arbitrary concurrent equivalence. Rebase merges have no calibrated component/toll model. General footprint profiling may be exponential; native validation may reveal different information.

One-workstation authored arrivals, instrumentation and cache/JIT effects limit timing inference; human benefit is unmeasured. Sparse gains, failed calibration and expensive compilation remain adverse. Finite tests and AI-assisted inspection are neither mechanical proofs nor independent reproductions.

10 Conclusion

Cached completion permits one policy to meet a universal call cap and the exact whole-kernel protection frontier for every hidden write bound. Partial repair and call costs create distinct information boundaries. Source checks and arrivals expose realizable count tradeoffs and limits on timing interpretation.

Data Availability

The public supplement [19] contains fixed inputs, sources, raw outcomes, corrections, expanded derivations and replay commands. All central theorem and proposition proofs appear in this paper; archived priced results are auxiliary. The zero-free evidence [18] is unchanged. Hashes record path projections. Modified sources retain notices; upstream archives, runtimes, caches, articles and private correspondence are excluded.

References

- [1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [2] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/pop109.pdf>.
- [3] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [4] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [5] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [6] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [7] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [8] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [9] Avi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [10] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [11] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [12] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [13] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [14] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [15] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [16] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [17] Krzysztof Postek, Ward Romeijnnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [18] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [19] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Version 19 one-job minimax regret and repair-work comparison, commit c49ae8b. <https://github.com/research-artifact-archive/research-artifact/tree/c49ae8b>.

- artifact/tree/c49ae8bc180c17f5283cef9444324d3de3457ae8.
- [20] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
 - [21] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint* arXiv:1804.00947v1. <https://arxiv.org/abs/1804.00947>.
 - [22] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651cc57834dbfcaa102ddfacae0d86cfb66/ sysdeps/unix/ sysv/linux/x86. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.
 - [23] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. c2_MacroAssembler_x86.cpp and globals_x86.hpp. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cfa0defb956e13a2c/src/hotspot/cpu/x86>.
 - [24] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
 - [25] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint* arXiv:2301.05089v2. <https://arxiv.org/abs/2301.05089>.
 - [26] Zvi Lotker, Boaz Patt-Shamir, and Dror Rawitz. 2012. Rent, Lease, or Buy: Randomized Algorithms for Multislope Ski Rental. *SIAM Journal on Discrete Mathematics* 26, 2, 718–736. <https://doi.org/10.1137/100794018>.
 - [27] Manish Purohit, Zoya Svitkina, and Ravi Kumar. 2018. Improving Online Algorithms via ML Predictions. In *NeurIPS* 31. <https://proceedings.neurips.cc/paper/2018/hash/73a427badebe0e32caa2e1fc7530b7f3-Abstract.html>.
 - [28] Thodoris Lykouris and Sergei Vassilvitskii. 2018. Competitive Caching with Machine Learned Advice. In *ICML*, PMLR 80, 3296–3305. <https://proceedings.mlr.press/v80/lykouris18a.html>.